

Is This Login Page Safe?

A simple security check of a sample login page — explained in plain language

I built a simple admin login page using HTML, CSS, and JavaScript, then checked it for common security mistakes — the kind that real beginner projects often have without realizing it. This document explains, in plain language, what I found, how I tested it, and how to fix it.

What I checked	A sample Admin Login page (index.html, dashboard.html, style.css)
Internship	CodeAlpha Cyber Security Internship — Task 3
How I checked	Read through the code, then tried to actually break it using Chrome's browser tools
What I found	5 security problems — 2 big, 2 medium, 1 small

What is this page supposed to do?

The page is a simple login form. You type in a username and password, and if they're correct, you're taken to a "Dashboard" page that says "Welcome, you are logged in as administrator."

Sounds simple, right? But while building it, I made a few mistakes that are actually very common — mistakes that let someone get into the dashboard without ever knowing the real password. Below, I'll walk through each problem one by one.

Quick Summary — What's Wrong?

#	The Problem	In Simple Terms	How Bad?
1	The password is written in plain text inside the code	Anyone can read it	Big Problem
2	The "are you logged in?" check happens only in the browser	Easy to trick	Big Problem
3	The search box lets you inject your own HTML into the page	Can be abused	Medium Problem
4	The password is literally hinted on the login page	No guessing needed	Medium Problem
5	Login status is stored in a cookie anyone can edit	Weak protection	Small Problem

The Problems, One by One

Bug #1: The Password is Written in Plain Text

Big Problem

Where is it?

Inside index.html, in the JavaScript code that checks your login.

What's actually happening?

When you click "Login", the page checks your username and password using a simple line of code, right there in the JavaScript. Since this code runs in your browser, it has to be sent to everyone who visits the page — including the password itself.

```
if (username === "admin" && password === "admin123") {  
    // let the user in...  
}
```

How I tested it

I right-clicked the login page and chose "View Page Source" (you can also press Ctrl+U). Within seconds, I could see the username ("admin") and password ("admin123") written in plain, readable text — no hacking tools needed at all.

Why this matters

Imagine writing your house key's shape on a sign outside your house. That's basically what's happening here. Anyone — even someone with zero hacking experience — can open the browser's "View Source" option and read the password directly.

How to fix it

Never check passwords using JavaScript in the browser. Passwords should be checked on a server, and stored using something called "hashing" (a way to scramble a password so even the website owner can't read the original) — never as plain, readable text.

Bug #2: The Login Check Only Happens in the Browser**Big Problem****Where is it?**

Inside dashboard.html, at the top of the page's script.

What's actually happening?

When you log in successfully, the page creates something called a "cookie" — a small piece of saved information — that says "loggedIn=true". Every time the dashboard page loads, it just checks: "Does this cookie say true?" If yes, it shows the protected content.

The problem is that cookies created this way (using JavaScript) can be created or changed by anyone, directly from their own browser — without ever typing a real password.

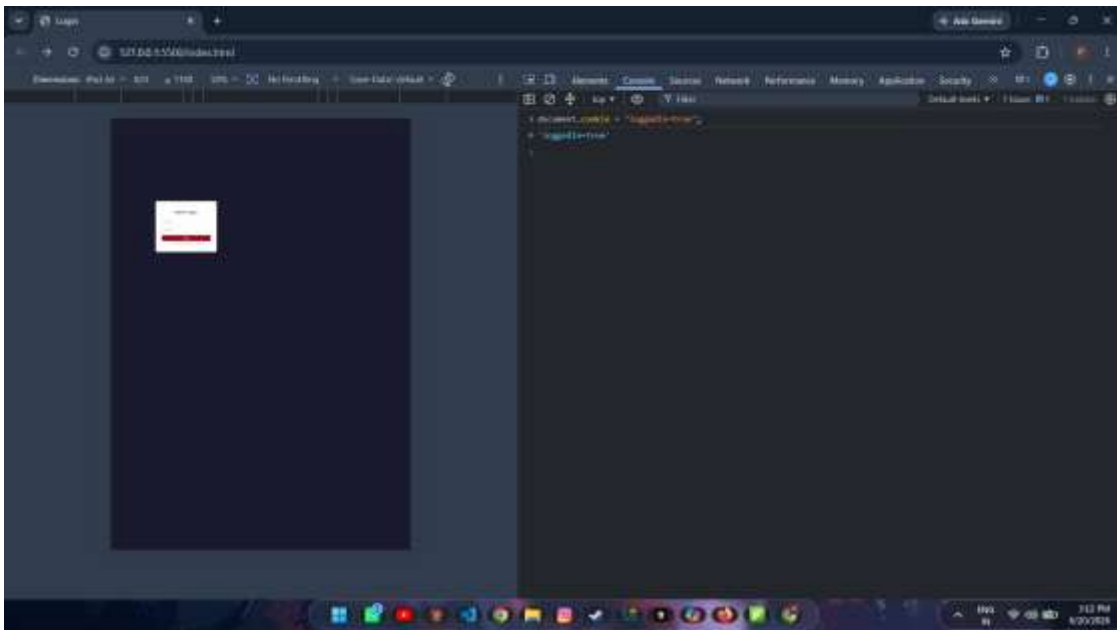
```
if (getCookie("loggedIn") !== "true") {  
    window.location.href = "index.html"; // not logged in, go back  
}
```

How I tested it (step by step)

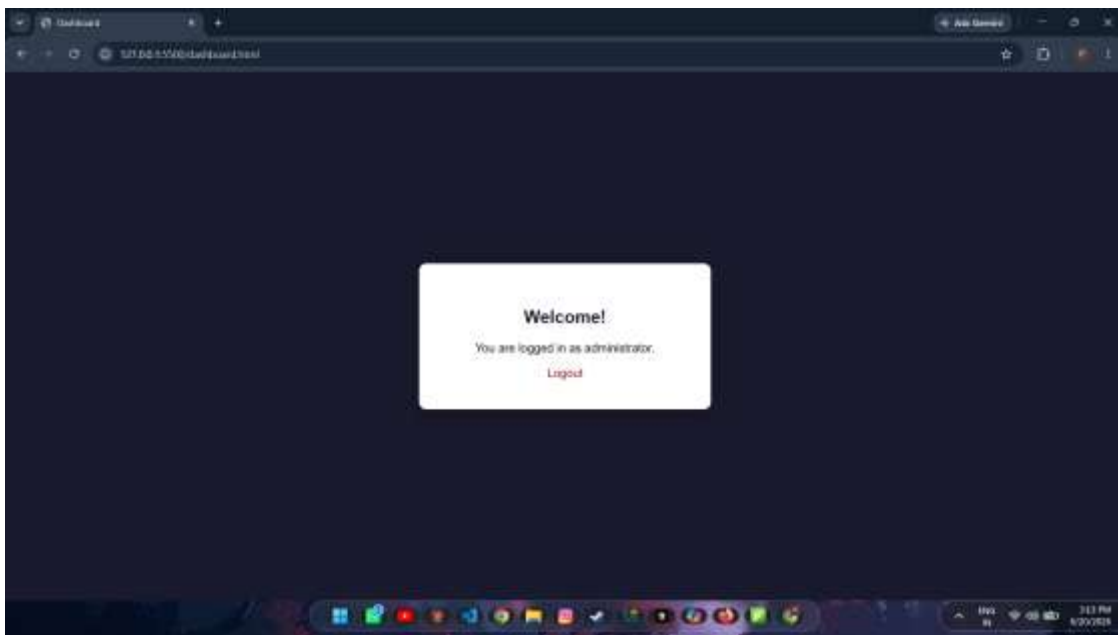
- Opened the login page, but didn't type anything in.
- Opened Chrome's Developer Tools (pressed F12) and clicked on the "Console" tab.
- Typed one line of code into the console:

```
document.cookie = "loggedIn=true";
```

- Then went directly to the dashboard page address — and it let me straight in. No password. No login form. Nothing.



Screenshot — Typing one line into the browser console, with no username or password entered anywhere.



Screenshot — And just like that, the protected dashboard opens up.

Why this matters

This is the same as a bouncer at a club who only checks if you're holding a wristband — but also lets you make your own wristband at home. The "security check" isn't really checking anything real.

How to fix it

A real server should be the one deciding if someone is logged in — not the browser. Logged-in status should be confirmed by the server every time someone tries to view a protected page, using a secure session method the browser can't fake.

Bug #3: You Can Inject Your Own Content Into the Page

Medium Problem

Where is it?

Inside dashboard.html — the part that reads the "search" text from the website address.

What's actually happening?

The dashboard page can show a little "search results" message based on text typed into the web address (the part after ?q=). The problem is, the page takes whatever is typed there and puts it straight onto the page — without checking what it actually is first.

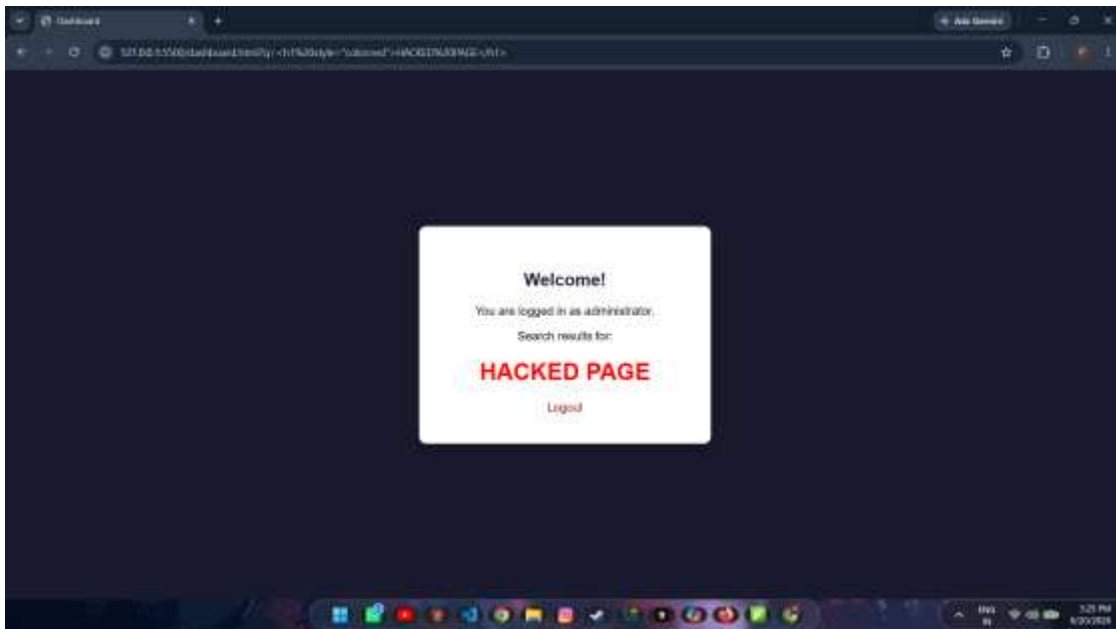
```
var searchTerm = params.get("q");
document.getElementById("searchResult").innerHTML =
    "Search results for: " + searchTerm;
```

How I tested it

I added this to the end of the dashboard's web address:

```
dashboard.html?q=<h1 style="color:red">HACKED PAGE</h1>
```

Instead of just showing that text, the page actually built it — a big red "HACKED PAGE" heading appeared right there on the dashboard.



Screenshot — The page actually built the HTML I put in the address bar, instead of just displaying it as text.

Why this matters

This is called "Cross-Site Scripting" (XSS) — a well-known type of security issue. If someone sent a link like this to a logged-in user, they could show fake content, fake login boxes to steal passwords, or quietly redirect the user somewhere dangerous.

How to fix it

Use a safer method called `textContent` instead of `innerHTML` when showing anything that comes from a search box, web address, or any kind of user input. `textContent` always shows things as plain

text — it never lets the content "build" as real HTML.

Bug #4: The Password Hint is Right There on the Page

Medium Problem

Where is it?

On the login page itself, just below the Login button.

What's actually happening?

The page shows this text to every single visitor:

```
"Hint: default admin password is admin123"
```

Why this matters

There's no need to guess or hack anything — the password is simply given away on the page. This is an easy mistake to make during development and forget to remove later.

How to fix it

Never show real passwords, hints, or default login details on any page that visitors can see — not even temporarily "just for testing." If a default account is needed for setup, force the person to change the password the first time they log in.

Bug #5: Login Info is Stored in a Cookie Anyone Can Edit

Small Problem

Where is it?

In both index.html and dashboard.html, wherever document.cookie is used.

What's actually happening?

Along with the "loggedIn=true" cookie, the page also stores "role=admin" the same way — using plain JavaScript. This is related to Bug #2, but worth mentioning separately: even if proper login checks were added on a server, storing things like "role" in a cookie that the browser can freely edit is still risky.

Why this matters

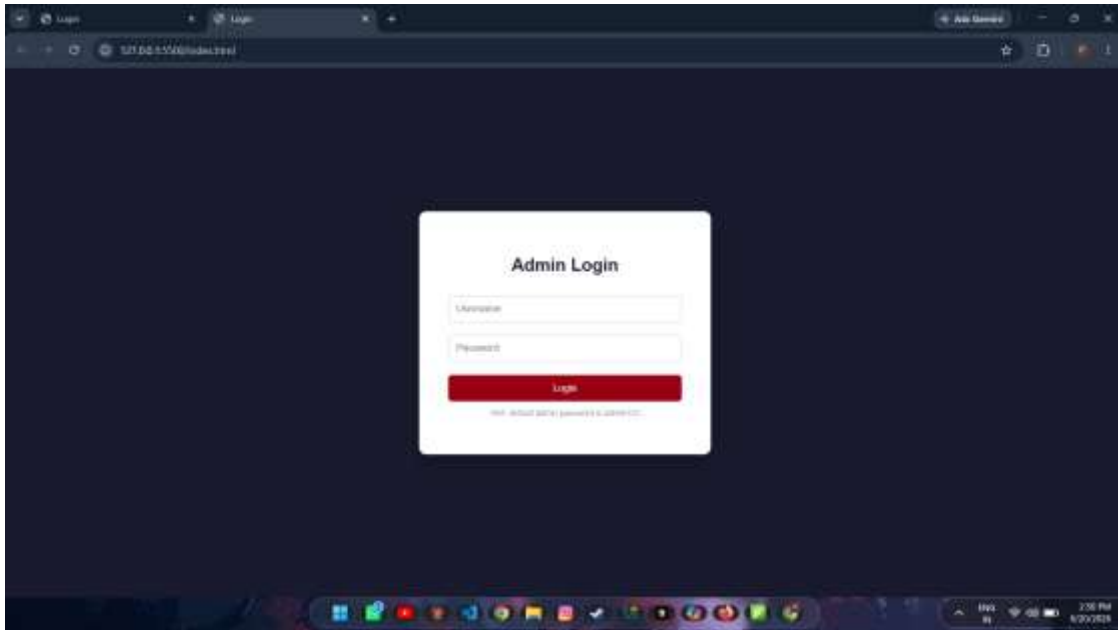
If a website ever has any other small security gap, an editable cookie like this makes it much easier for someone to give themselves "admin" access just by changing one value.

How to fix it

Mark important cookies as "HttpOnly" and "Secure" so JavaScript in the browser can't read or change them, and never store permission levels (like "admin") in a cookie the user's own browser can edit.

What the Login Page Actually Looks Like

For reference, here's the login page as a normal visitor would see it — notice the password hint right under the form, which is Bug #4 above.



Screenshot — The login page, password hint included.

If I Were to Fix This For Real...

Here's the order I'd fix things in, from most important to least:

1. Move the login check to a real server — the browser should never be the one deciding if a login is correct.
2. Store passwords safely on the server using hashing (e.g. bcrypt) — never compare plain-text passwords in JavaScript.
3. Use `textContent` instead of `innerHTML` for anything that comes from user input or a web address.
4. Set cookies as `HttpOnly` and `Secure` so they can't be read or changed from the browser console.
5. Remove any password hints, default passwords, or leftover testing notes before the page goes live.

What I Took Away From This

Going through this exercise made it really clear why front-end code (HTML, CSS, JavaScript that runs in the browser) should never be trusted to handle anything sensitive, like passwords or login checks. Anything that runs in someone's browser can be viewed, copied, or changed by that person — so real security always has to happen on a server that the user can't see or touch.

I also learned how small, easy-to-make mistakes (like a hint left on a page, or trusting a cookie without checking it) can completely break the security of an otherwise normal-looking login page.

Internship: CodeAlpha — Cyber Security Domain
Task: Task 3 — Secure Coding Review